



Introduzione a mongoDB

Python

MongoDB è un database NoSQL documentale, pensato per memorizzare e gestire dati in modo flessibile rispetto ai classici database relazionali.

A differenza di un database SQL tradizionale, dove i dati vengono organizzati in tabelle con righe e colonne, MongoDB salva le informazioni all'interno di documenti, cioè strutture simili al formato JSON.

Questa caratteristica rende MongoDB molto adatto quando i dati non hanno sempre una struttura fissa o quando le informazioni possono cambiare nel tempo.

Per esempio, in un'applicazione web potremmo avere utenti con dati diversi tra loro: alcuni potrebbero avere solo nome ed email, altri anche indirizzo, preferenze, ordini, storico attività o impostazioni personalizzate.



In MongoDB questi dati possono essere salvati con maggiore libertà, senza dover modificare continuamente lo schema del database.

Il concetto principale di MongoDB è il documento.

Un documento contiene coppie chiave-valore, per esempio nome, cognome, età, email o lista di acquisti.

Più documenti simili vengono raccolti dentro una collezione, che può essere vista come l'equivalente approssimativo di una tabella nei database relazionali.



Più collezioni fanno parte di un database.

MongoDB è molto usato nello sviluppo moderno perché si integra bene con applicazioni web, API, sistemi cloud e linguaggi di programmazione come JavaScript, Python, Java e C#, di base è particolarmente utile nei progetti in cui servono rapidità di sviluppo, scalabilità e gestione di dati semi-strutturati, come cataloghi prodotti, profili utente, log applicativi, contenuti dinamici, sistemi IoT e applicazioni real time.

Un altro punto importante è la sua capacità di scalare. MongoDB può essere utilizzato su una singola macchina, ma anche distribuito su più server per gestire grandi quantità di dati e molte richieste contemporanee.

Questo lo rende adatto sia a piccoli progetti sia ad applicazioni aziendali più complesse.



Nel contesto di Python, MongoDB viene usato tramite PyMongo, cioè il driver ufficiale che permette a un programma Python di collegarsi a MongoDB, creare documenti, leggerli, modificarli, eliminarli e fare interrogazioni più complesse.

PyMongo è indicato dalla documentazione MongoDB come il driver per applicazioni Python sincrone, mentre le operazioni CRUD principali includono inserimento, lettura, aggiornamento, sostituzione ed eliminazione dei documenti.



- Connessione al database

Questo codice rappresenta il punto di partenza.

MongoClient crea la connessione tra Python e MongoDB.

Successivamente si seleziona un database, in questo caso scuola, e una collezione, in questo caso studenti.

In MongoDB la collezione è simile, come idea generale, a una tabella SQL, ma contiene documenti flessibili invece di righe con schema rigido.

```
1.from pymongo import MongoClient
2.
3.# Creo la connessione al server
  MongoDB locale
4.client =
  MongoClient("mongodb://localhost:27
    017/")
5.
6.# Seleziono il database
7.db = client["scuola"]
8.
9.# Seleziono la collezione
10.studenti = db["studenti"]
```



- Inserimento di un documento

In Python un documento MongoDB viene rappresentato spesso come un dizionario.

Le chiavi del dizionario diventano i campi del documento, mentre i valori rappresentano le informazioni salvate.

Con `insert_one()` inseriamo un singolo documento nella collezione.

MongoDB assegna automaticamente un campo `_id` se non viene specificato manualmente.

La documentazione PyMongo indica `insert_one()` e `insert_many()` come metodi principali per inserire documenti.

```
1.from pymongo import MongoClient
2.
3.client =
  MongoClient("mongodb://localhost:27
    017/")
4.db = client["scuola"]
5.studenti = db["studenti"]
6.
7.# Creo un documento Python sotto
  forma di dizionario
8.nuovo_studente = {
9.  "nome": "Luca",
10.  "eta": 22,
11.  "corso": "Python",
12.  "voto": 28
13.}
14.
15.# Inserisco il documento nella
   collezione
16.risultato =
   studenti.insert_one(nuovo_studente)
17.
18.print("Documento inserito con ID:",
   risultato.inserted_id)
```



- Inserimento di più documenti

Quando bisogna salvare più elementi, si può usare `insert_many()`.

In questo caso Python invia a MongoDB una lista di dizionari.

Ogni dizionario diventa un documento separato dentro la collezione.

È una funzionalità utile quando si importano dati, si caricano record iniziali o si salvano più elementi generati da un programma.

```
1.from pymongo import MongoClient
2.
3.client =
  MongoClient("mongodb://localhost:27017/")
4.db = client["scuola"]
5.studenti = db["studenti"]
6.
7.# Lista di documenti da inserire
8.lista_studenti = [
9.    {"nome": "Anna", "eta": 24, "corso":
      "Data Analysis", "voto": 30},
10.   {"nome": "Marco", "eta": 21, "corso":
      "Python", "voto": 26},
11.   {"nome": "Giulia", "eta": 23, "corso":
      "Machine Learning", "voto": 29}
12.]
13.
14.# Inserisco più documenti insieme
15.risultato =
  studenti.insert_many(lista_studenti)
16.
17.print("ID inseriti:", risultato.inserted_ids)
```



- Lettura di un singolo documento

`find_one()` cerca un solo documento che rispetta il filtro indicato.

Il filtro viene scritto come dizionario Python.

In questo esempio MongoDB cerca il primo documento in cui il campo `nome` ha valore "Luca".

È utile quando ci aspettiamo un solo risultato, per esempio cercare un utente tramite email, un prodotto tramite codice o uno studente tramite matricola.

```
1.from pymongo import MongoClient
2.
3.client =
  MongoClient("mongodb://localhost:27017/")
4.db = client["scuola"]
5.studenti = db["studenti"]
6.
7.# Cerco il primo studente che ha nome Luca
8.studente = studenti.find_one({"nome": "Luca"})
9.
10.print(studente)
```



- Lettura di più documenti

`find()` restituisce tutti i documenti che rispettano una certa condizione.

A differenza di `find_one()`, può produrre più risultati, quindi viene spesso usato dentro un ciclo `for`.

La documentazione PyMongo indica `find()` e `find_one()` come metodi principali per recuperare documenti da una collezione.

```
1.from pymongo import MongoClient
2.
3.client =
  MongoClient("mongodb://localhost:27017/")
4.db = client["scuola"]
5.studenti = db["studenti"]
6.
7.# Cerco tutti gli studenti del corso
  Python
8.risultati = studenti.find({"corso":
  "Python"})
9.
10.# Stampo tutti i documenti trovati
11.for studente in risultati:
12.    print(studente)
```



- Filtro con operatori

MongoDB permette di usare operatori per creare filtri più potenti.

In questo caso `$gte` significa “greater than or equal”, cioè maggiore o uguale.

Quindi vengono cercati tutti gli studenti con voto almeno pari a 28.

Altri operatori comuni sono `$gt` per maggiore, `$lt` per minore, `$lte` per minore o uguale, `$ne` per diverso e `$in` per cercare valori contenuti in una lista.

```
1.from pymongo import MongoClient
2.
3.client =
  MongoClient("mongodb://localhost:27017/")
4.db = client["scuola"]
5.studenti = db["studenti"]
6.
7.# Cerco studenti con voto maggiore o uguale a 28
8.risultati = studenti.find({
9.  "voto": {"$gte": 28}
10.})
11.
12.for studente in risultati:
13.  print(studente)
```



- Aggiornamento di un documento

`update_one()` aggiorna il primo documento che corrisponde al filtro.

Il primo dizionario indica quale documento cercare, mentre il secondo indica quale modifica applicare.

L'operatore `$set` serve per impostare o modificare il valore di uno o più campi.

In questo esempio lo studente Luca riceve voto 30.

Questa funzionalità è utile per modificare dati già presenti senza dover cancellare e reinserire il documento.

```
1.from pymongo import MongoClient
2.
3.client =
   MongoClient("mongodb://localhost:27017/")
4.db = client["scuola"]
5.studenti = db["studenti"]
6.
7.# Aggiorno il voto dello studente Luca
8.risultato = studenti.update_one(
9.    {"nome": "Luca"},
10.    {"$set": {"voto": 30}}
11.)
12.
13.print("Documenti modificati:",
      risultato.modified_count)
```



- Aggiornamento di più documenti

`update_many()` permette di aggiornare tutti i documenti che rispettano una certa condizione.

In questo caso tutti gli studenti del corso Python ricevono un nuovo campo chiamato `stato`.

Questa operazione mostra bene la flessibilità di MongoDB: possiamo aggiungere un campo anche se i documenti precedenti non lo avevano.

```
1.from pymongo import MongoClient
2.
3.client =
   MongoClient("mongodb://localhost:27017/")
4.db = client["scuola"]
5.studenti = db["studenti"]
6.
7.# Aggiungo il campo "stato" a tutti gli
   studenti del corso Python
8.risultato = studenti.update_many(
9.    {"corso": "Python"},
10.    {"$set": {"stato": "attivo"}}
11.)
12.
13.print("Documenti modificati:",
       risultato.modified_count)
```



- Eliminazione di un documento

`delete_one()` elimina il primo documento che corrisponde al filtro indicato.

È importante usare sempre filtri precisi, perché un'eliminazione modifica realmente i dati presenti nel database.

Questo metodo si usa quando si vuole rimuovere un singolo elemento, per esempio un utente, un ordine o un record specifico.

```
1.from pymongo import MongoClient
2.
3.client =
  MongoClient("mongodb://localhost:27017/")
4.db = client["scuola"]
5.studenti = db["studenti"]
6.
7.# Elimino il primo studente con nome
  Marco
8.risultato = studenti.delete_one({"nome":
  "Marco"})
9.
10.print("Documenti eliminati:",
  risultato.deleted_count)
```



- Ordinamento dei risultati

`sort()` serve per ordinare i documenti restituiti da una query.

Il valore 1 indica ordine crescente, mentre -1 indica ordine decrescente.

In questo esempio gli studenti vengono ordinati in base al voto, dal più alto al più basso.

È utile quando vogliamo creare classifiche, report o visualizzazioni ordinate.

```
1.from pymongo import MongoClient
2.
3.client =
  MongoClient("mongodb://localhost:27017/")
4.db = client["scuola"]
5.studenti = db["studenti"]
6.
7.# Ordino gli studenti per voto dal più alto al
  più basso
8.risultati = studenti.find().sort("voto", -1)
9.
10.for studente in risultati:
11.    print(studente)
```



Buon Davante a tutti

